

Bandwidth-Effective DRAM Cache for GPUs with Storage-Class Memory

Jeongmin Hong[†], Sungjun Cho[†], Geonwoo Park[†], Wonhyuk Yang[†], Young-Ho Gong[★], and Gwangsun Kim[†]

POSTECH[†]

Department of Computer Science and Engineering
{jmhhh, allencho1222, geonwoo1998, wonhyuk, g.kim}@postech.ac.kr

Soongsil University[★]

School of Software
yhong@ssu.ac.kr

Abstract—We propose overcoming the memory capacity limitation of GPUs with high-capacity Storage-Class Memory (SCM) and DRAM cache. By significantly increasing the memory capacity with SCM, the GPU can capture a larger fraction of the memory footprint than HBM for workloads that mandate memory oversubscription, resulting in substantial speedups. However, the DRAM cache needs to be carefully designed to address the latency and bandwidth limitations of the SCM while minimizing cost overhead and considering GPU’s characteristics. Because the massive number of GPU threads can easily thrash the DRAM cache and degrade performance, we first propose an *SCM-aware DRAM cache bypass policy for GPUs* that considers the multi-dimensional characteristics of memory accesses by GPUs with SCM to bypass DRAM for data with low performance utility. In addition, to reduce DRAM cache probe traffic and increase effective DRAM BW with minimal cost overhead, we propose a *Configurable Tag Cache (CTC)* that repurposes part of the L2 cache to cache DRAM cacheline tags. The L2 capacity used for the CTC can be adjusted by users for adaptability. Furthermore, to minimize DRAM cache probe traffic from CTC misses, our *Aggregated Metadata-In-Last-column (AMIL)* DRAM cache organization co-locates all DRAM cacheline tags in a single column within a row. The AMIL also retains the full ECC protection, unlike prior DRAM cache implementation with Tag-And-Data (TAD) organization. Additionally, we propose SCM throttling to curtail power consumption and exploiting SCM’s SLC/MLC modes to adapt to workload’s memory footprint. While our techniques can be used for different DRAM and SCM devices, we focus on a *Heterogeneous Memory Stack (HMS)* organization that stacks SCM dies on top of DRAM dies for high performance. Compared to HBM, the HMS improves performance by up to $12.5\times$ ($2.9\times$ overall) and reduces energy by up to 89.3% (48.1% overall). Compared to prior works, we reduce DRAM cache probe and SCM write traffic by 91-93% and 57-75%, respectively.

I. INTRODUCTION

Rapidly-increasing data size in various domains [49], [115] has created huge challenges for the memory system of GPUs. Although High-Bandwidth Memory (HBM) has been adopted to meet the high memory bandwidth (BW) requirements of GPUs, it fails to fulfill the memory capacity needs of critical workloads, such as deep learning and large-scale graph analytics. Moreover, the memory capacity of GPUs has grown much slower than the compute throughput (Fig. 1a).

When data size exceeds GPU memory capacity, the data must be migrated repeatedly between the CPU and GPU, either manually or automatically. However, manual migration can be laborious for programmers, and it is infeasible for

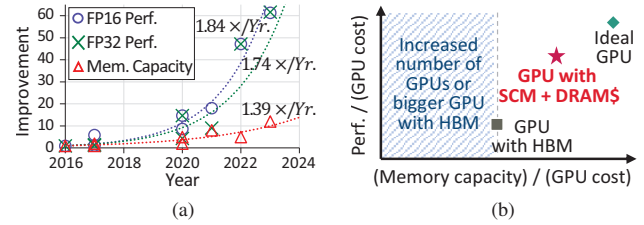


Fig. 1. (a) Improvement in compute throughput (with Tensor Core [2] and Matrix Core [7] when applicable) and memory capacity of GPUs over time [4], [5], [8], [11], [16]–[19]. (b) Cost-effectiveness of different GPU architectures under memory oversubscription.

irregular workloads because the data access pattern is unpredictable. On the other hand, demand paging approaches (e.g., NVIDIA Unified Memory [120]) can automatically manage data movement, but it can significantly degrade performance due to high page fault-handling latency and limited PCIe BW [46], [52], [121]. This overhead can be particularly severe for irregular workloads since prefetch/eviction policies become ineffective [14]. For example, the runtime of bfs can increase by $\sim 4.5\times$ with only 125% oversubscription (i.e., exceeding memory capacity by 25%) compared to when the GPU is not oversubscribed [47].

To avoid oversubscription, multiple GPUs can be used or bigger GPUs with more memory devices can be created. However, they superlinearly increase system cost due to high-speed link interface/switches [6] required and/or sublinear pin BW scaling with area [96]. Thus, these approaches lower memory capacity per GPU cost compared to the baseline GPU with oversubscribed HBM (shaded region in Fig. 1b). Using multiple GPUs can also require significant programmer efforts (§II-C).

Meanwhile, emerging Storage-Class Memory (SCM) offers a potential solution to the capacity limitations of DRAM with its higher memory density. Recent improvements in the device characteristics of the SCM, in terms of endurance [75], [76], reliability [56], performance [56], and capacity [141], have also made it an even more attractive choice. In addition, albeit slower than DRAM, SCM access is still much less expensive than accessing host memory through PCIe. SCM is also known to have a lower per-bit dollar cost than DRAM [104].

However, entirely replacing GPU’s DRAM with SCM

would be inefficient due to the lower performance and higher energy consumption of SCM [83], [131]. Thus, DRAM has to be used together to mitigate the disadvantages of SCM. In particular, HW-managed DRAM cache is suitable for GPUs as SW-managed scheme would incur high overhead from GPU page table update by the host-side driver [46].

To this end, we propose a novel DRAM cache design for GPUs with SCM. By significantly increasing the memory capacity with SCM, the GPU can avoid memory oversubscription entirely or capture a larger fraction of the memory footprint. At the same time, the performance impact of SCM is mitigated with an effective DRAM cache design. As a result, higher performance and memory capacity per cost can be achieved to approach the ideal GPU (Fig. 1b).

We design a *bandwidth-effective* DRAM cache optimized to meet GPU's high BW demands by minimizing the BW overhead from DRAM caches. In particular, a large number of concurrent memory accesses from 100,000s of threads can easily thrash the DRAM cache and waste BW for cache fills and write-backs. Prior work [25], [35], [147] on DRAM cache for CPUs proposed bypassing based on random sampling or access frequency for higher DRAM cache hit rate and lower latency. However, our DRAM cache requires a different bypass mechanism that considers GPU workload's access patterns (e.g., inter-thread spatial locality) and SCM properties, especially its long write latency and high write energy [83], [131]. In addition, simply maximizing DRAM cache hit rates may not enhance performance due to reduced parallelism across DRAM and SCM. Thus, we propose an *SCM-aware DRAM cache bypass policy for GPUs* that captures the multi-dimensional characteristics of SCM and GPU workload's access patterns (i.e., spatial locality, read/write access type, and access frequencies of pages) for effective caching.

Despite the bypass, excessive DRAM cache probe traffic can still result from tag accesses that contend with data accesses. To reduce the BW overhead, caching DRAM cache tags on-chip can be considered. However, blindly provisioning large amounts of SRAM to filter out tag accesses for large DRAM caches increases GPU cost without benefiting workloads that do not use the DRAM cache well (e.g., due to bypassing).

Thus, we propose a *Configurable Tag Cache (CTC)* to enable adjustment of SRAM capacity used for caching DRAM cache tags. The CTC repurposes some of the L2 cache ways to store DRAM cache tags. The user can configure the number of ways used for L2 cache and CTC, similar to configuration of L1 data cache and shared memory [8]. The CTC incurs low overhead by exploiting the existing L2 cache's data array.

However, when a CTC miss occurs, multiple tags of DRAM cachelines in a row need to be fetched. Thus, to minimize the DRAM BW overhead for tag accesses, we propose an *Aggregated Metadata-In-Last-column (AMIL)* organization that co-locates all tags from a DRAM row in the last column's data portion. The last column is used because it tends to be underutilized when data placement is done in an aligned manner. Although the SCM data that maps to the last column has to always bypass the DRAM cache, it accounts for a very

small fraction of data (e.g., only 1.56% of a 2048 KiB row with 32 B column of HBM) and incurs only 1.7% performance loss according to our study. The AMIL also retains the full ECC protection in the DRAM cache in contrast to prior work [35], [113], [125], [145] that has to repurpose ECC bits to store tags.

Among different approaches to combine SCM and DRAM for a GPU, we focus on the study of a *Heterogeneous Memory Stack (HMS)*, which integrates SCM and DRAM in a 3D-stacked memory using Through-Silicon Vias (TSV). As SCM and DRAM share the same bus in this design, the bus BW can be flexibly utilized across varying DRAM cache hit rates (§III-A). However, our DRAM cache is also effective even if SCM is integrated as separate devices or external SCM attached with high-speed links [10], [42].

We additionally propose power management and performance optimization to address SCM's device characteristics. When memory power consumption is high, the SCM can be throttled to reduce power by adjusting the timing parameters. Consequently, the HMS power can remain below the maximum power of an ideal high-capacity HBM, while still outperforming an oversubscribed HBM. In addition, when workload's footprint is small, the DRAM can be used as part of memory rather than a DRAM cache, to hold the majority of the data. The remaining data can be held in the SCM that operates in the performance-oriented SLC mode instead of capacity-oriented MLC mode. As a result, the performance of HMS can approach that of HBM for small memory footprint.

We demonstrate the effectiveness of the HMS using various GPU workloads that include multi-GPU large language model (LLM) training.

To summarize, we make the following contributions:

- **To the best of our knowledge, this work is the first to explore the design space of the DRAM cache for GPUs with SCM.** Our proposed GPU memory system can overcome the limited memory capacity and resulting performance degradation from oversubscription of DRAM-only GPUs.
- Our *Aggregated Metadata-In-Last-column (AMIL)* DRAM cache organization minimizes tag probe overhead by keeping all tags in a single row without compromising ECC protection as in prior works.
- We propose *SCM-aware DRAM cache bypass policy for GPUs* to minimize the performance penalty of SCM by considering the memory access patterns of GPUs and the device characteristics of DRAM and SCM.
- Our *Configurable Tag Cache (CTC)* repurposes a user-specified portion of the L2 cache ways to store DRAM tags, substantially reducing DRAM tag probe overhead.
- We show that our DRAM cache can significantly improve performance by up to $12.5\times$ ($2.9\times$ overall) and reduce energy consumption by up to 89.3% (48.1% overall) compared to HBM, with low hardware overhead.
- We propose simple techniques to mitigate SCM's power consumption and performance impact by adjusting the operation modes of the SCM and DRAM.

II. BACKGROUND AND MOTIVATION

A. Unified Memory

Modern GPUs support Unified Memory (UM) [120] that provides a single virtual address space for the host and device and automates data transfers between them without explicit copies. UM also enables GPU memory oversubscription, allowing kernel's memory footprint to exceed GPU memory capacity. It is especially useful for large irregular workloads under oversubscription (Fig. 2a), as manual memory copy is infeasible for unpredictable access patterns.

When a GPU accesses data in the host memory, a page fault occurs to initiate page transfers or swaps between the host and device, in 4 KiB page granularity on x86 [15]. This process involves CUDA runtime and GPU driver on the host, and the data transfer goes through PCIe with limited BW, leading to low performance [15], [121]. To recover performance, prior works [46], [73], [90] proposed prefetch, eviction, and data transfer schemes. For example, Tree-Based Neighborhood (TBN) prefetch and pre-eviction policies of NVIDIA GPUs adaptively migrate data in larger granularity of up to 1 MiB for high PCIe BW utilization [46]. The vDNN [117] exploits the access patterns of activations known a priori for prefetching and eviction in DNN training. However, their effectiveness can be limited for irregular workloads due to unpredictable access patterns. Some recent GPUs [9] support host connectivity through NVLink with a high BW of 900 GB/s, but oversubscription still hurts performance as we show in §IV-B.

B. Modeling Unified Memory

As computer architecture research is often done using simulators, prior work on HW-assisted UM [45], [46] modeled UM by modifying GPGPU-sim [23]. However, due to the slowdown from page faults by the GPU, the simulation speed is also slowed down significantly (up to $5\times$ in our evaluation) by oversubscription. Based on our estimation, **simulating a full A100 GPU (80 GiB) that is oversubscribed to hold 75% of the memory footprint would take up to 57 years.** Consequently, to our knowledge, all prior work on UM used scaled-down configurations for simulation, using footprints between 15-74 MiB on average [46], [73], [79], [90], [156]. To validate the methodology, we analyzed the impact of oversubscription on a real NVIDIA RTX 2080 Ti GPU and a simulated GPU [45] for representative workloads. For the real GPU, we induced oversubscription by pinning dummy data on the GPU, thereby limiting available memory to 75% of the workload's memory footprint. Input data were generated using [26], [30]. Results in Fig. 3 show that real GPU exhibited similar or even higher slowdown from oversubscription than in simulation. This discrepancy can be attributed to the simulator's optimistic page-fault handling latency of $20\mu s$, which is known to be a lower bound [73], [156]. Also, larger footprints under the same oversubscription ratio further slowed down the real GPU. The simulation results for oversubscribed GPU are also consistent with measurements on real GPU [48], [121]. Although not shown here due to space constraints, compute-bound workloads tested – 2mm [51] and lavamd [30] – also

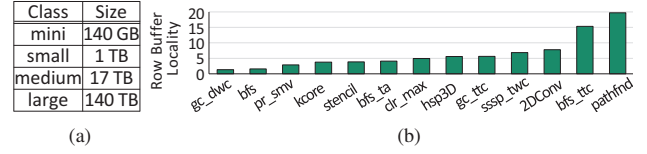


Fig. 2. (a) Graph500 benchmark's data size example [1]. (b) Row buffer locality (defined as the average number of column accesses per row activation) of representative workloads.

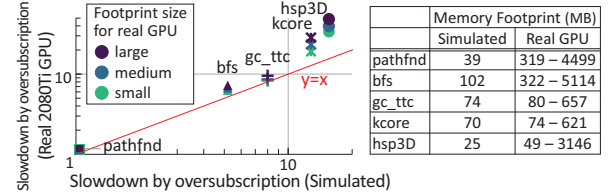


Fig. 3. (Left) Validation of UM simulation at a fixed oversubscription ratio (log scale plot). (Right) Workloads' memory footprints used for validation.

showed the same behavior, with little slowdowns. Thus, we adopt the simulator to model UM.

C. Challenges of Multi-GPU Programming

If a workload's data do not fit in a GPU, multiple GPUs can be used to partition the data. However, currently CUDA or OpenCL cannot automatically scale a single-GPU workload to multiple GPUs. Thus, in general, the programmer has to manually modify the code to split the data and computation even for regular workloads. Moreover, additional kernels often have to be created to process data shared between GPUs and communication has to be manually optimized for best performance [103]. For irregular (e.g., graph) workloads, the code often has to be entirely rewritten using frameworks for the target domain (e.g., WholeGraph [139] for GNN and Pangolin [31] for graph pattern mining). Due to the overhead of graph pre-processing, load-imbalance, and inter-GPU communication, the performance often scales poorly with GPU count and may even be degraded [21], [63], [109]. Therefore, reducing the number of GPUs for large-scale workloads by increasing GPU memory capacity can often not only reduce system cost but also improve performance. In addition, higher GPU memory capacity widens the range of workloads that a single GPU can execute.

D. SCM Characteristics

SCM refers to a set of non-volatile memory (e.g., Phase Change Memory or PCM) located between DRAM and flash devices in the memory hierarchy in terms of latency, BW, and density. PCM uses phase-change material that switches between a high-resistance amorphous state (logical "0") and a low-resistance crystalline state (logical "1") [83] and is mature enough to be commercialized [41], [134]. For state transition, the cell is heated up to crystallization (melting) point for a SET (RESET) operation. PCM can also provide Multi-Level Cell (MLC) capability [83] and multiple decks in a die for higher capacity [41], [141]. Moreover, PCM can realize high 10-year

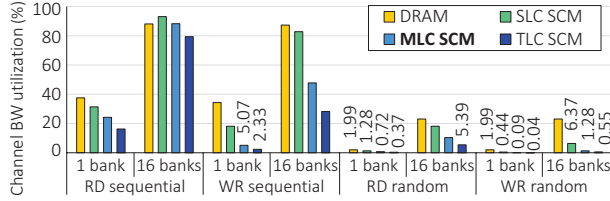


Fig. 4. Memory channel BW utilization from memory devices with HBM organization for synthetic access patterns (configuration details in §IV-A).

data retention temperature [135]. Recent SCM also uses less power than conventional 3D XPoint memory [56] and several works showed high endurance of $10^{11} - 10^{12}$ programming cycles [75], [76].

Although SCM has longer row activation latency, the column access latency (i.e., t_{CL}) is the same as that of DRAM since the row buffer access mechanism is orthogonal to memory technology [83], [131]. Thus, when the row buffer locality is high, even slow memory devices can saturate the memory channel. In addition, current DRAM devices, such as HBM and GDDR, have channel BW significantly lower than the internal BW from multiple banks in a die. Thus, even if each bank's BW is lowered by replacing the DRAM arrays with SCM arrays, the channel BW can still be saturated with high row buffer locality.

Synthetic traffic results in Fig. 4 show that, for sequential read accesses over 16 banks, different SCM devices can achieve similar channel BW utilization as DRAM, even though single-bank BW of SCM is considerably lower than that of DRAM. The SLC SCM even achieves slightly higher BW than DRAM by eliminating refresh operations. Recent work [141] also demonstrated that SCM can provide a high capacity of 256 Gib (cf. 24 Gib DDR5 DRAM chip [82]) while providing high 15 GB/s BW from a single chip, although its interface was not disclosed (cf. 51.2 GB/s peak BW from 8 chips in a consumer DDR5-6400 DIMM). However, for streaming writes, high SCM write latencies result in lower overall BW even with 16 banks. Furthermore, for random accesses, SCM BW reduces further due to very low locality.

While Optane DIMM with PCM exhibits low BW even for streaming accesses [61], it can result from its multiple levels of SRAM and DRAM buffers within the DIMM, internal address translation, and intra-PCM data migration that can severely degrade PCM performance [84], [133], rather than the raw performance of PCM. Using PCMCsim [84] and synthetic streaming access pattern, we confirmed that, without such overhead, the PCM chip's $2 \times$ DDR4-2666 interface BW can be saturated. LENS [133] also reported a consistent result (4KiB data access from PCM in 100ns, achieving ~ 40 GB/s BW).

E. Considerations in DRAM Cache Design for GPU with SCM

While GPUs require high memory BW, SCM throughput varies substantially based on access type (i.e., read or write) and locality (Fig. 4). GPU workloads also have varying access locality (Fig. 2b). Thus, characteristics of GPU workloads

and SCM should be carefully considered for DRAM cache to hold hot data with low spatial locality, while cold data with high spatial locality resides in SCM. By filtering writes to SCM, DRAM cache can also mitigate the high write latency and energy [131]. However, choosing which data to cache in DRAM is challenging because of multi-dimensional access characteristics (i.e., spatial locality, hotness, and write intensity).

Most prior work on DRAM cache targeted CPUs and focused on minimizing latency [35], [64], [113], [124], [143], [148], so they can be suboptimal for GPUs, which are more sensitive to memory BW than latency [22]. In addition, prior works on bandwidth-efficient DRAM cache assumed on-package DRAM cache backed by off-package DRAM [35], [143], [148], rather than DRAM cache backed by SCM that we assume. Furthermore, DRAM caches are often managed in page granularity [13], [66], [70], [88], [114], [133], [138], [148], [155], but GPUs can suffer from the resulting waste of BW. Also, the spatial locality that GPU exhibits *across threads* in a warp or thread block is different from the *intra-thread* spatial correlation in CPU workloads from complex data structures or control-flow [126]. Thus, footprint caching [62], [64], [65] proposed for CPUs can be ineffective for GPUs. We discuss prior work further in §V.

To our knowledge, no prior work incorporated spatial locality across GPU threads in designing the DRAM cache as we do in determining the DRAM cache design, bypass policy, and on-chip DRAM cache metadata caching mechanism.

F. Feasibility of TSV-based 3D Stack of SCM and DRAM

In TSV-based 3D integration, each die is fabricated separately, using different processes if needed. It avoids the manufacturing difficulties of sequentially fabricating a top die directly on a bottom die in monolithic 3D [50]. 3D-stacking of heterogeneous dies with TSV has been extensively studied and demonstrated for PCM [97], DRAM [3], CMOS sensors [53], flash devices [85], and MEMS [132]. Here, we examine key considerations regarding the feasibility of HMS.

In general, TSVs can pose signal integrity issues due to coupling between a TSV and nearby TSVs or circuitry, as well as reliability issues arising from the high-temperature manufacturing process [97]. The mass production of HBMs since 2015 [99] demonstrates that these challenges have been well understood and overcome for 3D-stacked DRAM. Because the HMS places TSVs in the same peripheral IO circuitry region as in HBM (Fig. 5a) [67], apart from the memory cell array, we do not introduce any new challenges in these aspects compared to HBM. In addition, SCM media, such as typical PCM with ovonic threshold switch (OTS) [32], [141], are compatible with back-end-of-line (BEOL) process and can withstand high-temperature TSV fabrication process [59].

Additionally, power delivery network (PDN) with TSVs [97] should provide sufficient power for SCM. Considering $\sim 10\times$ difference in access *energy* between DRAM (~ 1 pJ/bit [28], [108]) and PCM (~ 10 pJ/bit [60], [83]) and that PCM accesses are 10-100x slower than

DRAM [60], [83], [131], PCMs can consume similar or less power than DRAM per bank (i.e., power=energy/delay) [83]. However, compared to DRAM, multiple SCM row accesses from more banks can overlap due to its longer delays, using more power per channel. Recent DRAM with processing-in-memory capability has shown that 4-5x higher power can be supplied within HBM [86] and that t_{FAW} constraint can be removed [81]. Thus, the PDN issue can be addressed similarly for SCM.

In addition, heat dissipation from SCMs in an HMS can be an issue for temperature-sensitive DRAM [93]. It is a fundamental challenge in 3D stacks, including HBMs, and HBMs can be throttled by the memory controller at high temperatures [12]. Similarly, we show that a simple SCM throttling technique can effectively mitigate the thermal issue (§III-E), and even without throttling, the worst-case peak HMS temperature differs from that of HBM by less than 0.1% (§IV-E) as SCMs are placed in the upper rank of the stack, close to the heat sink. Recent cooling solutions (e.g., liquid immersion cooling adopted in production datacenters [20], [119]) are also proven to be more energy-efficient while allowing processors to operate at higher power in comparison to air cooling. Thus, they can allow for more aggressive SCM devices. The energy and heat issues of SCM can also be mitigated by device scaling because the energy of SCMs, such as PCM, decreases with the cell material volume [43], [137].

III. DRAM CACHE FOR GPUS WITH SCM

A. Design Space of Heterogeneous Memory

To improve GPU's memory capacity under fixed pin BW, the DRAM cache and SCM can be integrated in a 3D-stack to create a Heterogeneous Memory Stack (HMS) shown in Fig. 5a or as separate memory devices (Fig. 5b). The separate SCM devices can also be attached using external NVLink [42] or CXL [10]. The designs differ in how the devices are mapped to memory channels. In HMS, the DRAM and SCM can share the same channel as different ranks (Fig. 6a) similar to [136], [143], whereas separate devices inevitably use separate buses (Fig. 6b) similar to [64], [65], [125], [144].

However, for flexible channel BW utilization for varying traffic patterns, each channel should be shared by both the DRAM cache and SCM. For example, if a workload shows a high DRAM cache hit rate, the SCM-only channel in Fig. 6b can become idle while the DRAM-only channel experiences high contention, resulting in only 50% utilization overall. In contrast, in Fig. 6a, both channels can be fully utilized by the DRAM caches in each channel (Fig. 6c). Optane DIMM is also placed on the same channel as DRAM DIMM [133].

Thus, we focus on the HMS design with DRAM and SCM ranks sharing the same channel implemented using TSVs in a 3D stack, but we show that our DRAM cache is also effective for SCM integrated with separate channels (§IV-B). HMS retains the HBM's high-level design and interface, including TSV connectivity, the number of banks and bank groups, and the base die's I/O buffers for signal integrity [68]. The key difference is replacing the upper-rank DRAM dies in HBM

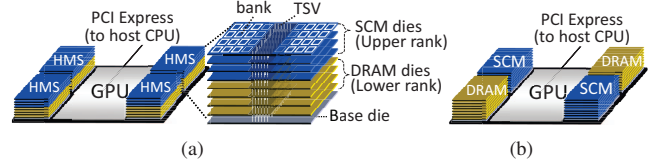


Fig. 5. Design space of a GPU with SCM and DRAM cache with (a) 3D-stacked DRAM and SCM and (b) separate DRAM and SCM stacks.

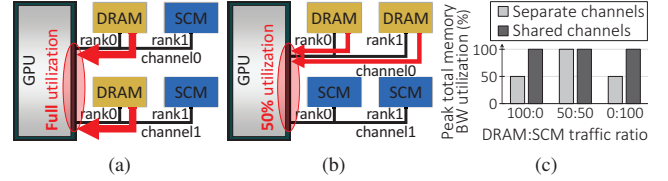


Fig. 6. Integration of SCM and DRAM cache using (a) shared channels and (b) separate channels. (c) Peak memory BW for varying DRAM-to-SCM traffic ratios.

with SCM dies in HMS (Fig. 5a). Thus, each channel has a DRAM cache rank and an SCM rank. Although the DRAM cache is not addressable by the programmer, HMS has a larger addressable capacity than HBM due to SCM's higher bit density; HMS provides $2\times$ addressable memory capacity than HBM, assuming SCM has $4\times$ bit density compared to DRAM [33], [114]. We use PCM as SCM due to their maturity [134], but other SCM devices can also be used.

Due to GPU's high memory BW demand, maximizing the effective BW is a key consideration for our DRAM cache. HW-managed DRAM caches for CPUs typically use 64 B cachelines [25], [61], [95], [113]. In contrast, in this work, we assume a 256 B DRAM cacheline¹ to achieve high memory bus utilization, amortize the long activation latency of SCM, and exploit the high spatial locality of memory accesses from GPUs. In addition, to reduce the BW overhead of fetching DRAM cache tags (hereafter, tags) and metadata (e.g., LRU bits), we make the DRAM cache direct-mapped and reduce the tag size. Combined with the large cacheline size, the small tag size enables capacity-effective on-chip tag caching (§III-D) that further reduces the BW overhead of DRAM cache probes. To track DRAM cache misses, we use SRAM-based MSHR for each channel of the DRAM cache located near the memory controller. DRAM cache operations (e.g., probe, fill, and eviction) are translated by a DRAM cache controller into DRAM or SCM requests, which are then scheduled by the memory controller, considering timing parameters.

B. Aggregated Metadata-In-Last-Column (AMIL)

Our CTC (§III-D) keeps all tags of a DRAM cache row in a single L2 cache sector to exploit the high spatial locality of GPU workloads. Thus, we propose AMIL to minimize the tag access overhead by fetching all tags in a row with a single column access. Prior work on DRAM cache with conventional

¹For L1 and L2 caches, we still assume 128 B line with 32 B sectors [72].

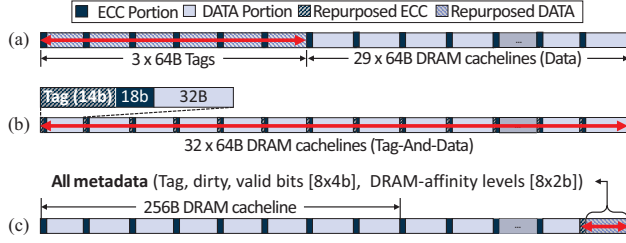


Fig. 7. DRAM cache row with (a) Loh-Hill cache [95], (b) Tag-and-data (TAD) [113], [125], and (c) proposed Aggregated Metadata-In-Last-column (AMIL). The double arrows indicate the columns with tag information.

cacheline sizes [95] proposed a highly set-associative (e.g., 29-ways) organization that places tags in the first few columns and data in the remaining columns, requiring multiple columns to be accessed to fetch all tags in a row as shown with a double arrow in Fig. 7a. Alloy cache [113] proposed a direct-mapped DRAM cache that fetches a Tag-And-Data (TAD) with a single access (Fig. 7b). However, TAD distributes the tags across all columns, so the entire row has to be accessed to fetch all tags. In addition, to comply with DRAM standards, DRAM caches with TAD [25], [35], [113], [143]–[146] have to repurpose some ECC bits to store tags, degrading reliability.

To minimize BW overhead, AMIL places all metadata (tags, valid, dirty, and DRAM-affinity bits described in §III-C2) of a row in the last column's 32 B data portion (Fig. 7c). Although the last column cannot be used to cache SCM data, it accounts for a very small fraction (only 1.6% for a 32 B column in a 2 KiB row) of a row. Thus, AMIL effectively overcomes the reliability limitation of prior DRAM caches based on TAD.

The AMIL is enabled by the high DRAM/SCM capacity ratio and large cacheline size we propose, unlike prior DRAM caches [35], [95], [113], [143], [144] with a few GiBs of DRAM cache for 10s-100s of GiBs of main memory. Assuming SCM has 4× the capacity of a DRAM die and using a direct-mapped DRAM cache, the DRAM cache tag is 2-bit. With valid/dirty bits and 2-bit DRAM-affinity, each cacheline only requires 6-bit metadata. With the 256 B DRAM cacheline and a 2 KiB row, each row includes 8 cachelines, needing only 48 bits for metadata. This metadata is also protected with ECC.

C. SCM-aware DRAM Cache Bypass Policy

Our SCM-aware DRAM cache bypass policy considers the multi-dimensional characteristics of accesses (§II-D), i.e., spatial locality, hotness, and write intensity, to keep useful data in DRAM and avoid DRAM cache thrashing from 100,000s of GPU threads. The key insight is that we can quantify the combined effects of these three-dimensional characteristics with a one-dimensional score metric. First, our *SCM penalty score* accounts for the spatial locality and write intensity by comparing the latency penalty of SCM versus DRAM for given requests. Then, the score is multiplied by hotness (i.e., per-page activation counter) to obtain the final *DRAM-affinity score*. The scores can be calculated during runtime at a low cost (§III-F), without any separate profiling phase.

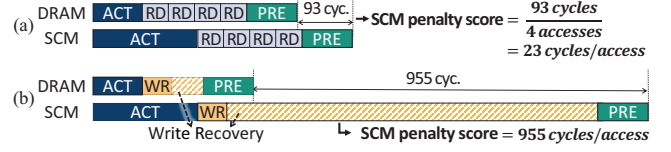


Fig. 8. Example timing diagrams contrasting the SCM penalty score calculated when a row is accessed with (a) multiple read accesses and (b) a single write access. Not drawn to scale.

1) *SCM Penalty Score*: The SCM penalty score reflects the *latency penalty of SCM per column access*. High-penalty accesses cache data in DRAM, while others bypass the DRAM cache to access SCM directly. The score considers the spatial locality within the row buffer and differentiates writes from reads. The spatial locality is essential to consider, as memory accesses with many row buffer hits can amortize the long SCM activation latency. Consequently, accessing such data from SCM has a lower performance impact than when few row buffer hits occur. On the other hand, write-intensive data should be cached in DRAM because the write latency is higher for SCM than for DRAM and SCMs can have limited write endurance [131].

For the bypassing decision, the latency of memory accesses to an SCM row is first calculated based on timing parameters for required operations such as row activation, column accesses, write recovery, and precharge. Similarly, the latency required to serve the same memory accesses from DRAM (i.e., as if they were all accessed from DRAM) is also calculated. The difference between these two latencies is then divided by the number of column accesses to obtain SCM's per-access penalty.

$$SCMPenaltyScore = \frac{Latency_{SCM} - Latency_{DRAM}}{NumColumnsAccessed} \quad (1)$$

For example, when there are no writes and SCM's long activation delay is well amortized over multiple column accesses, the SCM penalty score is low (Fig. 8a). In contrast, when there is a write without spatial locality, the latency discrepancy between SCM and DRAM is large, and the SCM penalty score is high (Fig. 8b). With the scores, our policy (§III-C3) can bypass the DRAM cache for the access pattern in Fig. 8a and cache data accessed in Fig. 8b. Thus, DRAM contention can be reduced while keeping data in DRAM when it is beneficial.

The SCM penalty score can be computed at a low cost. Because column access latency is identical between SCM and DRAM [131], it will be canceled out in the numerator of Eq. 1. Thus, the numerator can be approximated and statically pre-computed as $(t_{RCD,SCM} - t_{RCD,DRAM})$ if the accesses include only reads or as $(t_{RCD,SCM} - t_{RCD,DRAM} + t_{WR,SCM} - t_{WR,DRAM})$ if writes are included. Then, it is simply divided by the number of columns accessed, which can be recorded in the DRAM cache's MSHR along with the presence of write. This implementation requires two 32-bit registers for the pre-computed values and an ALU.

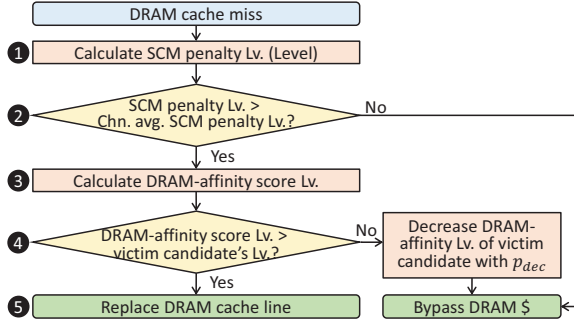


Fig. 9. SCM-aware DRAM cache bypass policy.

2) *DRAM-Affinity Score*: The SCM penalty score incorporates the spatial locality and presence of writes but ignores data's access frequency, which requires historical information and cannot be inferred from current requests. Thus, we propose another score metric called *DRAM-affinity score*, calculated by multiplying a request's SCM penalty score with its per-page *activation counter*. The activation counter is incremented when a DRAM or SCM row is activated. The DRAM-affinity score is discretized into N_{levels} levels with a fixed interval and kept in the DRAM cache as metadata (Fig. 7(c)) for bypass policy.

3) *SCM-aware DRAM Cache Bypass Policy*: Because accessing the victim DRAM cacheline's DRAM-affinity level for every DRAM cache miss would incur very high BW overhead, we propose a two-level bypass policy to minimize this overhead. The first-level comparison is done to filter the majority of the requests without any DRAM BW overhead. If the comparison is passed, the second-level comparison is done using the victim's metadata in DRAM.

First, as shown in Fig. 9, when a DRAM cache miss occurs, the SCM penalty score for the requests mapped to the same row is calculated and discretized to N_{levels} levels between 0 and the maximum value observed so far (1). This discretization prevents inconsistent bypass decisions due to small fluctuations in the score. The discretized score is then compared to a similarly-discretized moving average of the SCM penalty score maintained by the memory controller (2). If the request's score level is less than or equal to the average level, the DRAM cache is bypassed (i.e., no miss fill is done).

Otherwise, the current request's DRAM-affinity level is compared with the victim cacheline's DRAM-affinity level (3, 4). If the current request has a higher level, the victim is replaced (5), and the affinity level is stored. If the victim is invalid, the miss fill is done without this comparison. If the replacement is not done, the victim's score level is decremented with a probability p_{dec} to adapt to changing working set. p_{dec} is calculated as the accessed page's activation counter divided by the maximum activation counter observed by this memory controller. The intuition is that the victim's DRAM-affinity level should be more likely to be decremented if hot data bypassed the DRAM cache. Score calculations can be done with an FPU with six 32-bit registers to hold average, maximum, and current request values for SCM-penalty score

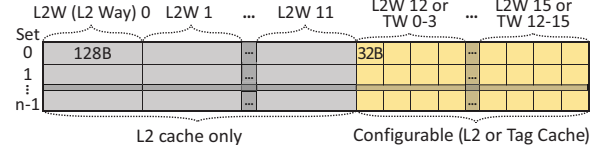


Fig. 10. Configurable Tag Cache in L2 cache, assuming the Tag Cache (TC) can use up to 4 L2 cache ways and each L2 cache way can hold 4 TC ways.

and DRAM-affinity scores. The activation counters can be tracked in 2 MiB granularity with low overhead as 160 GiB GPU memory requires only 80 KiB from 80-kilo entries of 8 bits counters. To address counter saturation, a 3-bit register indicates the position of the MSB bit to implement low-cost right-shifts. The LSB bits can be zeroed over time and ignored until the shifts are finished. Depending on the workload's characteristics, the activation counters can be used selectively (e.g., use a constant value of 1 if its benefit is not high).

D. Configurable Tag Cache (CTC)

Our bypass policy reduces DRAM traffic for DRAM cache misses. However, determining a hit or miss also requires a tag access from DRAM [113] for every L2 cache miss, incurring high overhead. Using an additional on-chip *tag cache* to hold DRAM cacheline tags [35] can incur high overhead for our DRAM cache, due to its significantly larger capacity. The MissMap [95] approach statically partitions the LLC to hold only DRAM cache presence information with low overhead, instead of the full tags. However, such static partitioning can degrade the performance when high LLC capacity is required. For example, recent GPUs support L2 cache resident control, whereby programmers can specify some data to persist in the L2 cache for high performance [130]. To meet the varying demands of workloads, flexible partitioning is necessary.

Thus, we propose a *Configurable Tag Cache (CTC)* to reduce the DRAM cache probe traffic and support flexible partitioning between the L2 cache and tag cache without a separate SRAM for caching tags (Fig. 10). The programmer can specify the number of tag cache ways out of the total L2 cache ways, similar to how the user chooses the split between the L1 data cache and shared memory [8]. In general, workloads with high DRAM BW demand can benefit more from additional CTC ways, as a CTC miss generates DRAM cache probe traffic that contends with the demand DRAM traffic. The number of CTC ways can also be determined by profiling [80] or set-dueling [112]. For iterative workloads [30], [106], [129], it can be changed across kernels by flushing dirty lines, but we leave such a study for future work. If DRAM is configured as part of memory [125], all ways are used for L2 cache.

A single L2 cache way is divided into four 32 B Tag Cache ways, assuming a 128 B L2 cacheline. The size of the DRAM tags for a row is 4 B, excluding the DRAM-affinity bits not kept in CTC (§III-B). Thus, a Tag Cache line is further divided into eight 4 B sectors that are mapped to eight DRAM rows. To minimize area overhead, we assume that up to four L2 cache ways can be used for tag caching. The CTC requires

modification of L2, but its overhead is low as it adds only $8+8+22=38$ bits (per-sector valid and dirty bits, and per-line tag) per cacheline and 4-bit pseudo-LRU metadata per set. The storage overhead is 612 bits per set or only 2.5% of L2 cache.

E. Power Management and Performance Optimization

Accessing SCM cells can require higher energy and power consumption than DRAM, leading to higher temperatures. Especially, SCM power consumption needs to be managed for HMS that stacks SCM on DRAM. Thus, we propose a simple SCM power throttling technique that monitors the memory stack's temperature [3] and adjusts SCM's timing parameters. If the temperature increases too high, the timing parameters for SCM activation (t_{RCD}) and/or write recovery (t_{WR}) are doubled to limit power consumption. In our evaluation, throttling is rarely required, but it can effectively curtail SCM's power and temperature increase if needed.

In addition, when the memory footprint is small (e.g., based on the memory allocation for UM), GPU's DRAM can be used as part of memory along with SCM, rather than a cache. For high performance, data can initially be placed in DRAM, with the remaining data mapped to SCM. Additionally, SCM can operate in SLC mode, instead of MLC mode, for enhanced performance. As a result, the GPU can minimize performance impact for small workloads and our evaluation results show that HMS can provide high performance for varying memory footprints.

F. Putting It All Together

The operations of our DRAM cache can be summarized as follows. When an L2 cache miss occurs, the CTC is first looked up. If a CTC hit, it is immediately determined whether the request hits the DRAM cache. If not, the DRAM cache must be probed to access the tag and fill CTC. With AMIL, tags for the entire row are fetched with a single DRAM access, amortizing the probe overhead for subsequent accesses. If a DRAM cache hit occurs, the request accesses DRAM and the average SCM penalty score is updated. Otherwise, the requested address is first accessed from SCM to serve the demand access, and then, the DRAM cache bypass policy (§III-C3) determines if the DRAM cache fill should be done. With 128 B L2 cacheline and 32 B sectors, all L2 fills are done in 32 B size whether it is fetched from DRAM cache or SCM, whereas data movement between DRAM and SCM uses 256 B DRAM cacheline size.

IV. EVALUATION

A. Methodology

We integrated Accel-sim [72] with a UM model [46] and Ramulator [77] for simulation (Table I). Due to the very long simulation time of the oversubscribed baseline (§II-B), we downscaled an NVIDIA A100 GPU by 1/5 while keeping constant ratios between SM count, L2 cache capacity, memory

TABLE I
SIMULATED SYSTEM CONFIGURATION.

SMs	
21 SMs, 64 warps/SM, 65536 regs/SM, clock frequency: 901 MHz L1+shared memory: 192 KiB/SM, 128 B line (32 B sectors), LRU L1 SRAM latency and BW: 15 cycles and 17 GB/s/SM	
L2 cache and CTC parameters	
L2(Baseline): 128 B line (32 B sectors), 16 ways, 8 MiB capacity, LRU L2(HMS): 128 B line (32 B sectors), 12 ways, 6 MiB capacity, LRU CTC(HMS): 32 B line (4 B sectors), 16 ways, up to 2 MiB capacity, LRU Freq: 901MHz, latency:120 cycles, peak BW: 402GB/s from 16 banks	
Memory organization (for both DRAM and SCM)	
row buffer: 2 KiB, bus width: 128 bit (BL 2, DDR), # of channels: 8, # of dies: 8, # of bank groups per ch.: 4, # of banks per bank group: 4, FR-FCFS scheduler Bus frequency: 1 GHz, Bus peak BW: 256 GB/s from 8 channels	
Timing parameters	
DRAM [77]	CL: 14, RCD: 14, RAS: 33, WR: 16, RP: 14 (row hit:15ns, row miss(closed page):43ns)
SCM [74], [131]	CL: 14, RCD: 120, RAS: 120, WR: 1000, RP: 14 (row hit:15ns, row miss(closed page):149ns)
Unified Memory-related latency and BW	
PCIe link	BW: 12.8 GB/s (i.e., 1/5 of PCIe 4.0 $\times 16$) or 64 GB/s (§IV-C)
NVLink	Latency for CPU memory access (cacheline size): 0.135 μ s [47] (where applicable) BW: 76.8 GB/s (CPU memory BW of 46.6 GB/s)
Other	Page fault handling latency: 20 μ s [73]
Memory energy (pJ/bit) [83], [140]	
DRAM	ACT: 1.17, PRE: 0.39, RD: 0.93, WR: 1.02
SCM	ACT: 2.47, PRE (WR): 16.82, RD: 0.93, WR: 1.02

channel count, and PCIe (or NVLink) lanes.² In addition, we also show results using the full 64 GB/s PCIe BW (§IV-C). We used AccelWattch [69] to model GPU energy and 8 pJ/bit PCIe or NVLink energy [37].

We used 22 workloads [29], [30], [51], [106], [129] with memory footprints ranging from 19 to 135 MiB (68 MiB on average), excluding those with smaller footprints. We define R_{HBM} as the relative capacity of HBM compared to the memory footprint and assume $R_{HBM}=75\%$ (i.e., HBM holds 75% of the workload's memory footprint) unless otherwise stated. To model oversubscription, we adjusted HBM's capacity (i.e., the number of page frames available) as in all prior works [46], [73], [79], [90], [156] for simulation feasibility (§II-B). Other memory stacks were also configured to have the same capacity per DRAM die, and $4\times$ capacity per SCM die compared to a DRAM die. For instance, for a 100 MiB workload, HBM has a 75 MiB capacity while the DRAM cache and SCM have 37.5 MiB and 150 MiB capacities, respectively. We also evaluated SCM-only 3D-stack ("SCM") and an ideal HBM ("Infinite HBM" or "InfHBM") with unlimited capacity (i.e., never oversubscribed). The SCM timing parameters we assume are conservative, considering real SCM device [141] has demonstrated shorter latencies. The SCM energy parameters are also conservative as we assume a higher energy than the energy reported in a recent study of SCM [128].

For our DRAM cache, we focus on HMS due to its high speedups but also present results with separate DRAM/SCM

²Simulating a single workload took up to 24 days even with the downscaling. To simulate a full A100 GPU, the workloads' problem sizes needed to be scaled up accordingly to prevent a significant portion of the memory footprint from fitting in the 40 MiB L2 cache, which would substantially increase the simulation time.

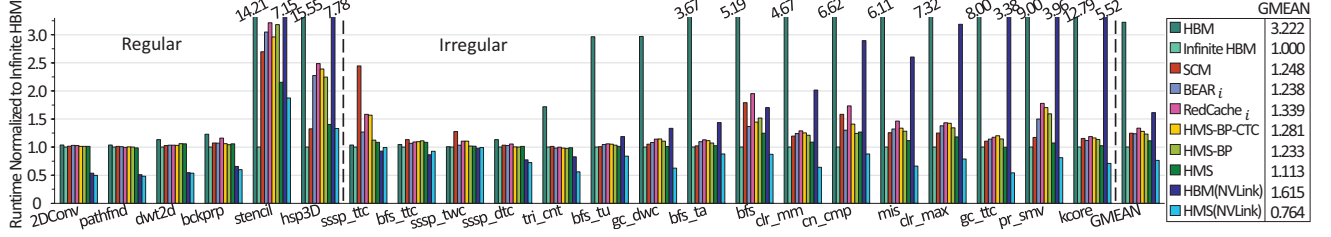


Fig. 11. Runtime of GPUs with different memory designs normalized to Infinite HBM. PCIe was used for host connectivity unless otherwise stated.

buses (§IV-B). We assumed $F_{update}=100$ and $N_{levels}=4$ for HMS. For the moving average, a new value has a weight of 1%. We disabled the activation counter for simplicity although an ideal activation counter's speedup is up to 7.6% (0.4% overall). To understand the impact of each technique, we also evaluated HMS without bypass and CTC (HMS-BP-CTC or HMS-B-C) as well as HMS without bypass (HMS-BP or HMS-B). For conservative evaluation of CTC, its size was reduced to hold only a quarter of the total tags in the DRAM cache and ranged between 1-4 KiB across workloads. The total DRAM cache tags of a full A100 GPU that replaces 40 GiB HBMs with equivalent HMSes is 40 MiB – equal to the L2 cache capacity. Thus, we configure the CTC to use a quarter of the 16 L2 ways to hold a quarter of all DRAM cache tags.

We also evaluated prior works on BW-efficient DRAM caches (with 64 B DRAM cachelines) adopted for the DRAM cache within HMS. For BEAR [35], we modeled an ideal DRAM Cache Presence bit such that the DRAM cache presence is known without LLC lookup or DRAM cache probe overhead, and refer to it as BEAR_i; for its Neighboring Tag Cache, we assumed the same 704 B/channel as in [35]. For RedCache [25], we assumed an ideal gamma update without DRAM BW overhead and refer to it as RedCache_i. For the mostly-clean DRAM cache [124], we assumed a perfect cache predictor and zero-cost tag probes, referring to it as McCache_i.

We assumed input data were initially in host memory, and we used the TBN prefetcher and pre-emption policies for UM [46] (§II-A), which migrate data in 4 KiB to 1 MiB granularity adaptively, as in NVIDIA GPUs.³ We also studied replacing PCIe with high-BW NVLink for host connectivity. We kept the BW ratios of CPU/GPU memory and NVLink the same as in NVIDIA Grace Hopper Superchip [9] (Table I). We modeled the dynamic access counter scheme for NVLink, which considers the amount of free memory capacity and access frequency to migrate hot pages to the GPU while cold data is accessed directly from the remote memory in cacheline granularity [47], [120]. For several plots, we only show representative workloads due to space constraints, but the average values reported are always calculated over all workloads.

³Using a first-touch policy instead of the NVIDIA UM scheme significantly degraded performance by 2.75× overall.

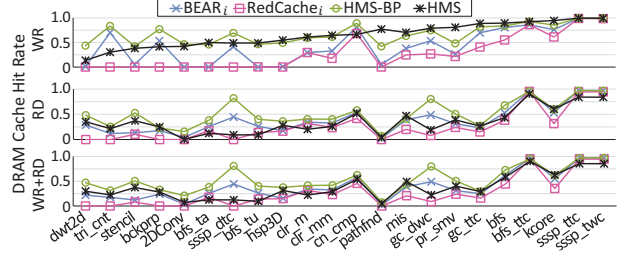


Fig. 12. DRAM cache hit rates with different designs.

B. Performance

Compared to the oversubscribed HBM, HMS can hold the entire memory footprint and achieved a significant speedup of up to 12.5× (2.9× overall) by reducing data transfers over PCIe by up to 159× for stencil (7.3× on average) (Fig.11). The speedup was especially pronounced for graph workloads with irregular access patterns, for which UM page prefetchers are ineffective. Despite having a smaller DRAM capacity than HBM, our DRAM cache effectively filters out requests to SCM. For example, SCM resulted in up to 2.25× longer runtime than HBM for *sssp_ttc*, as the SCM was frequently accessed with little row buffer locality for writes. In contrast, HMS reduced its performance impact using the DRAM cache with write hit rates of 99.6% (Fig. 12). Because *sssp_ttc* has a relatively smaller working set per kernel, it did not suffer significantly from oversubscription with HBM. For some graph workloads (e.g., *bfs_tu*, *bfs_ta*, *gc_**, *clr_**, etc.) with a relatively higher row buffer locality and/or low write-intensity, DRAM cache hit rates for HMS were relatively low at 10-30% due to bypass, but write requests still had high hit rates of 49-89%. For some workloads with high row buffer locality and read-intensity, SCM achieved similar performance as InfHBM as the long activation latency was amortized.

For regular workloads, HBM's performance varied depending on the working set size and the effectiveness of the UM prefetcher. While it had similar performance as InfHBM for some workloads (e.g., *pathfind* and *2DConv*), it suffered significantly for others (e.g., *stencil* and *hsp3d*). In contrast, HMS reduced the performance gap between the HBM and InfHBM from 15.55× (14.21×) to 1.40× (2.15×) for *hsp3d* (*stencil*) with higher capacity. Overall, HMS outperformed HBM and SCM by 2.9× and 12.1% on average, respectively, achieving within 11.3% of the performance of the InfHBM.

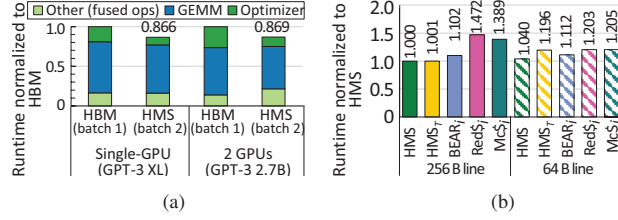


Fig. 16. (a) GPT model [27] training time with HBM and HMS using TensorFlow XLA. (b) Performance impact of DRAM cacheline size (64 B vs. 256 B) with different DRAM caches. HMS_T is a variant of HMS with TAD instead of AMIL.

C. Sensitivity Study and Additional Results

DRAM cacheline size impact. Compared to using 64 B line, 256 B line provided 4% speedups overall for HMS due to improved CTC caching and the amortization of SCM latency (Fig. 16b). HMS_T benefited more from 256 B line (19.4% speedup overall), as TAD generates more DRAM traffic than AMIL from a CTC miss, and a larger line results in fewer tags. BEAR_i improved little with 256 B line (0.9% on average) while the performance of RedCache_i and McCache_i degraded with 256 B line, as they are unaware of SCM and increased SCM traffic further. In addition, for HMS, reducing cacheline size from 256 B to 128 B degraded performance by up to 11% (1.7% overall), while increasing it to 512 B had a negligible impact. 1 KiB cacheline degraded performance by up to 6% due to increased data movement.

Memory footprint impact. Even for workloads with relatively small memory footprints, HMS showed competitive performance (within 1%) compared to HBM by using DRAM as a part of memory and SCM in SLC mode (Fig. 17a).⁴ As the relative memory footprint increases, HMS can use the SCM in TLC mode for higher capacity, achieving even greater speedups (up to 52.3× for *sssp_dtc*). Even when the HMS was oversubscribed with a relative footprint of 4.0, it still outperformed HBM by up to 108.8× (2.85× overall) by reducing page faults. Thus, HMS can better serve diverse workloads than HBM. Additionally, for varied R_{HBM} , HMS also consistently outperformed Bear_i (RedCache_i), by up to 62% (87%), except for *bokprp* for which Bear_i outperformed HMS by only 1.2% (Fig. 17b).

CTC and AMIL sensitivity. To analyze the impact of CTC size and AMIL, we varied the number of L2 cache ways used for CTC from 4 to 1 for AMIL and TAD (Fig. 18). We assumed the same 256 B DRAM cacheline size to isolate its effect. With AMIL, reducing the CTC size by a quarter had a low performance impact of only 1.5% overall, whereas TAD showed higher performance impact of 5.9%. AMIL outperforms TAD in handling the increased CTC miss, as AMIL needs a single DRAM access for a CTC miss, whereas TAD needs eight accesses due to distribution of tags in a row. As a result, TAD_CTC1 resulted in up to 5.6× (2.6× overall) more DRAM accesses than AMIL_CTC1.

⁴For SLC (TLC) SCM, we assumed RCD = 60 (250), RAS = 60 (250), and WR = 150 (2350) cycles [131].

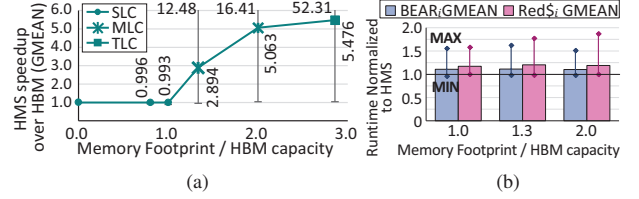


Fig. 17. (a) Speedup with HMS over HBM for varying relative memory footprint over HBM capacity. The symbols in the legend indicate the mode of HMS's SCM among SLC, MLC, and TLC. Error bars show the maximum and minimum across workloads. (b) Runtime of BEAR_i and RedCache_i normalized to HMS for varying relative memory footprint for all workloads.

DRAM vs. SCM capacity ratio impact. We evaluated the effects of different capacity ratios between DRAM and SCM by varying their row counts. For a configuration of 2 SCM dies and 6 DRAM dies ("2SCM-6DRAM"), runtime increased by up to 12.4× for *kcore* (2.9× overall), and energy increased by 1.99× compared to 4SCM-4DRAM due to smaller GPU memory capacity and frequent page faults. 6SCM-2DRAM showed 6.5% (10.4%) higher runtime (energy) than 4SCM-4DRAM, as the DRAM cache hit rate decreased by 20.6% overall due to smaller DRAM capacity. For example, *hsp3D*'s DRAM cache hit rate fell by 49% and the SCM activation increased by 35%, resulting in a 29% increase in runtime.

PCIe BW and other sensitivities. With 64 GB/s PCIe BW, HMS still outperformed HBM and SCM by 2.21× and 16.28% overall, respectively. HMS also still outperformed BEAR_i (RedCache_i) by 16.13% (19.89%) overall. Increasing N_{levels} from 4 to 8 slightly degraded performance by 0.3% overall due to increased traffic to probe victim's DRAM affinity level.

D. Energy and Power

HMS substantially reduced energy consumption by up to 89.3% (48.1% overall) compared to HBM (Fig. 19⁵) by reducing data movement and runtime. Compared to SCM, HMS also considerably reduced energy by up to 68.0% (16.5% overall) as our DRAM cache effectively mitigated the high energy cost of SCM accesses. SCM-agnostic BEAR_i and RedCache_i did not measurably reduce energy, consuming 10.7% and 76.8% more energy on SCM access than HMS, respectively. They also consumed 74.7% and 7.4% more energy on DRAM access overall than HMS due to frequent DRAM cacheline movements and tag probes. HMS(NVLink) also reduced the energy by up to 80.1% (22.7% overall) compared to HBM(NVLink).

While SCM can increase power usage, our simple SCM throttling technique (§III-E) can effectively prevent the power usage of HMS from exceeding the maximum power of HBM (Fig. 20). In our evaluation, *stencil* showed the highest power consumption for InfHBM. While HMS without throttling consumed more power for *stencil*, using throttling effectively reduced the power to 54.5% below that of InfHBM. It resulted in 55% performance loss but still outperformed the baseline

⁵In AccelWattch [69], "Static" refers to energy from leakage currents of inactive components, and "Const" refers to peripheral component energy such as GPU board fans and other auxiliary circuitry.

comparison to Loh-Hill cache [95]. Timber [102] and AT-Cache [58] proposed a fixed-size on-chip SRAM storage for DRAM cache tags while our CTC allows size configuration by users. ACCORD [144] mitigated high BW overhead of set-associative DRAM caches by coordinating way install and prediction. Tag Tables [44] modifies page table to compress DRAM cache tags and cache them in LLC. Footprint-based DRAM caches [62], [64], [65] exploits *intra-thread* spatial locality of CPU threads (§II-E). BEAR [35] addressed DRAM cache's BW bloat with probabilistic bypassing, write probe filtering with metadata in LLC, and fetching/caching neighbor DRAM cacheline tags for demand accesses. RedCache [25] bypasses DRAM cache with dynamic access count thresholds to identify hot data. DICE [146] is a dynamic cacheline indexing scheme for compressing DRAM caches. Baryon [91] uses compression and sub-blocking to efficiently utilize fast memory capacity with low BW overhead. These compression schemes can be adopted in our DRAM cache to further improve effective BW. Several works [57], [79], [145] proposed DRAM cache for remote data in other GPUs or CPUs. PoM [123] and CAMEO [34] proposed using stacked DRAM to expand address space, rather than as a cache. Page-granularity DRAM cache management has also been proposed [88], [101], [148]. Sim et al. [124] proposed keeping DRAM cache mostly clean. However, they did not consider SCM's characteristics.

B. Hybrid and Adaptive Memory Hierarchy

Several prior works [114], [142], [154] proposed memory systems with DRAM and PCM, optimizing for page management and endurance. Yoon et al. [142] and Zhao et al. [155] proposed data placement mechanisms considering row buffer miss frequency similar to our hotness metric, but they did not consider the inter-thread spatial locality of GPUs. They also managed the DRAM cache in a large row-granularity, but smaller DRAM cachelines are more effective for GPUs (§IV-C). 3D-Xpath [87] proposed a 3D memory stack combining density-optimized and performance-optimized DRAM. Memory hierarchies using sub-ranks [118] or sub-channels [28] can improve energy efficiency by finer granularity accesses. These approaches are orthogonal to our design and can be combined. Ohm-GPU [151] proposed a silicon photonics-based optical network for GPU, DRAM, and 3D XPoint memory for high BW. However, our work focuses on a more practical near-term solution. ZnG [150] and FlashGPU [152] integrated flash devices in the memory hierarchy, which can be effective for read-intensive, regular access patterns. However, for frequent irregular writes, the high ($\sim 100\mu s$) write latency and granularity would require an effective DRAM cache. Our DRAM cache design can be adopted in their designs to improve performance. Thermostat [13] is a SW scheme to manage data placement between SCM and DRAM under user-specified performance constraints. Kim et al. [78] and Liu et al. [94] also proposed hybrid memory systems managed by the OS. However, for GPU workloads, SW-managed DRAM caches can become a

bottleneck. Several works [55], [133], [136] analyzed Optane PM DIMM and proposed SW optimizations to improve performance. Optane DIMM does not necessarily represent the SCM we assume, as it includes a hierarchy of buffers that access 3D XPoint memory with a large 4 KiB granularity. Moreover, requests can be reordered by an on-DIMM queue. Recent key-value stores [39], [40], [127] exploited heterogeneous memory hierarchy. Harmony [92] proposed scheduling of tasks and data movement for training large DNNs on a GPU. GPM [110] exploits the persistency of CPU-attached Optane from GPU. MMS [111] proposed HW/OS support for adapting between high-density and low-latency PCM modes. Power token [54] was proposed to manage PCM power with fine-grained write.

VI. CONCLUSION

We propose an effective DRAM cache for GPUs with SCM to overcome the memory capacity wall while achieving high memory BW. Our AMIL organization fetches all tags in a row with a single access to reduce the tag probe BW overhead, while retaining full ECC protection in contrast to prior DRAM caches with TAD organization. Furthermore, to prevent DRAM cache thrashing from a massive number of threads while considering the characteristics of SCM, we propose an *SCM-aware DRAM cache bypass policy*. This policy leverages the SCM penalty score and DRAM-affinity score, which captures the multidimensional characteristics of access patterns (i.e., access frequency, row buffer locality, and write intensity) in a single score, for simple yet effective bypassing. In addition, because DRAM cache probe traffic can interfere with data access from DRAM, we propose CTC to reduce the probes with little overhead while enabling flexible capacity adjustment between CTC and L2 cache. Consequently, we reduce DRAM cache probe and SCM write traffic by 91-93% and 57-75%, respectively, over prior works. Our SCM throttling can effectively curtail SCM power usage below the maximum HBM power while still achieving high speedups over HBM. Using SCM's SLC and MLC modes, the GPU can also adapt to workload's memory footprint and performance demand. The results show that our proposed GPU with SCM and DRAM cache significantly outperforms the oversubscribed baseline GPU with HBM by up to $12.5\times$ ($2.9\times$ overall).

ACKNOWLEDGMENT

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grants (No.2021-0-00871, Development of DRAM-Processing-In-Memory Chip for DNN Computing, and No.2021-0-00310, Development of SW Framework for Server to Improve AI Training/Inference Efficiency) and National Research Foundation of Korea (NRF) grants (RS-2023-00277080 and RS-2023-00212711) funded by the Korea government (MSIT). We would like to thank the anonymous reviewers for their constructive comments. Gwangsun Kim is the corresponding author.

REFERENCES

- [1] "Graph500 Benchmark specification." [Online]. Available: https://graph500.org/?page_id=12
- [2] "Nvidia tensor cores." [Online]. Available: <https://www.nvidia.com/en-us/data-center/tensor-cores>
- [3] "High bandwidth memory (hbm) dram," JEDEC Standard, 2013.
- [4] "Nvidia tesla p100," NVIDIA whitepaper, 2016.
- [5] "Nvidia tesla v100 gpu architecture," NVIDIA whitepaper, 2017.
- [6] "Nvidia nvswitch: The world's highest-bandwidth on-node switch," NVIDIA Whitepaper, 2018.
- [7] "Introducing amd cdna architecture," AMD whitepaper, 2020.
- [8] "Nvidia a100 tensor core gpu architecture," NVIDIA Whitepaper, 2020.
- [9] "Nvidia grace hopper superchip architecture," 2020. [Online]. Available: <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper>
- [10] "Compute express link specification 3.0," CXL Consortium, 2022.
- [11] "Nvidia h100 tensor core gpu architecture," NVIDIA whitepaper, 2022.
- [12] "High bandwidth memory (hbm2e) interface intel agilex® 7 m-series fpga ip user guide," July 2023. [Online]. Available: <https://cdrdv2-public.intel.com/781867/ug-773264-781867.pdf>
- [13] N. Agarwal and T. F. Wenisch, "Thermostat: Application-transparent page management for two-tiered main memory," in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.
- [14] T. Allen and R. Ge, "Demystifying gpu uvm cost with deep runtime and workload analysis," in *Proceedings of the 33rd International Parallel and Distributed Processing Symposium (IPDPS)*, 2021.
- [15] T. Allen and R. Ge, "In-depth analyses of unified virtual memory system for gpu accelerated computing," in *Proceedings of the 34th International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.
- [16] AMD, "Amd instinct™ mi100 accelerator." [Online]. Available: <https://www.amd.com/en/products/server-accelerators/instinct-mi100>
- [17] AMD, "Amd instinct™ mi250x accelerator." [Online]. Available: <https://www.amd.com/en/products/server-accelerators/instinct-mi250x>
- [18] AMD, "Amd instinct™ mi300x accelerator." [Online]. Available: <https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html>
- [19] AMD, "Amd Radeon Instinct™ mi25 accelerator." [Online]. Available: <https://www.amd.com/ko/products/professional-graphics/instinct-mi25>
- [20] V.-G. Anghel, "Exploring current immersion cooling deployments," March 2023. [Online]. Available: <https://www.datacenterdynamics.com/en/analysis/exploring-current-immersion-cooling-deployments/>
- [21] A. Azad, M. M. Aznaveh, S. Beamer, M. Blanco, J. Chen, L. D'Alessandro, R. Dathathri, T. Davis, K. Dewese, J. Firoz, H. A. Gabb, G. Gill, B. Hegyi, S. Kolodziej, T. M. Low, A. Lumsdaine, T. Manlaibaatar, T. G. Mattson, S. McMillan, R. Peri, K. Pingali, U. Sridhar, G. Szarnyas, Y. Zhang, and Y. Zhang, "Evaluation of graph analytics frameworks using the gap benchmark suite," in *2020 IEEE International Symposium on Workload Characterization (IISWC)*, 2020, pp. 216–227.
- [22] A. Bakhoda, J. Kim, and T. M. Aamodt, "Throughput-effective on-chip networks for manycore accelerators," in *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*, 2010.
- [23] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *Proceedings of the 2nd International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009.
- [24] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," in *Proceedings of the 14th Transactions on Architecture and Code Optimization (TACO)*, 2017.
- [25] P. Behnam and M. N. Bojnordi, "Redcache: Reduced dram caching," in *Proceedings of the 57th Design Automation Conference (DAC)*, 2020.
- [26] G. Boeing, "Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.
- [27] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Proceedings of the 33rd Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [28] N. Chatterjee, M. O'Connor, D. Lee, D. R. Johnson, S. W. Keckler, M. Rhu, and W. J. Dally, "Architecting an energy-efficient dram system for gpus," in *Proceedings of the 23rd International Symposium on High Performance Computer Architecture (HPCA)*, 2017.
- [29] S. Che, B. M. Beckmann, S. K. Reinhardt, and K. Skadron, "Pannotia: Understanding irregular gpgpu graph applications," in *Proceedings of the 16th International Symposium on Workload Characterization (IISWC)*, 2013.
- [30] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *Proceedings of the 12th International Symposium on Workload Characterization (IISWC)*, 2009.
- [31] X. Chen, R. Dathathri, G. Gill, and K. Pingali, "Pangolin: An efficient and flexible graph mining system on cpu and gpu," *Proc. VLDB Endow.*, vol. 13, no. 8, p. 1190–1205, apr 2020.
- [32] W.-C. Chien, C.-W. Yeh, R. L. Bruce, H.-Y. Cheng, I. T. Kuo, C.-H. Yang, A. Ray, H. Miyazoe, W. Kim, F. Carta, E.-K. Lai, M. J. Brightsky, and H.-L. Lung, "A study on ots-pcm pillar cell for 3-d stackable memory," *IEEE Transactions on Electron Devices*, vol. 65, no. 11, pp. 5172–5179, 2018.
- [33] J. Choe, "Intel's 2nd generation xpoint memory - will it be worth the long wait ahead?" 2021. [Online]. Available: <https://www.techinsights.com/blog/memory/intels-2nd-generation-xpoint-memory>
- [34] C. C. Chou, A. Jaleel, and M. K. Qureshi, "Cameo: A two-level memory organization with capacity of main memory and flexibility of hardware-managed cache," in *Proceedings of the 47th International Symposium on Microarchitecture (MICRO)*, 2014.
- [35] C. Chou, A. Jaleel, and M. K. Qureshi, "Bear: Techniques for mitigating bandwidth bloat in gigascale dram caches," in *Proceedings of the 42nd International Symposium on Computer Architecture (ISCA)*, 2015.
- [36] E. Choukse, M. B. Sullivan, M. O'Connor, M. Erez, J. Pool, D. Nellans, and S. W. Keckler, "Buddy compression: Enabling larger memory for deep learning and hpc workloads on gpus," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- [37] B. Dally, "GTC china 2020 keynote," NVIDIA GPU Technology Conference, 2020. [Online]. Available: [https://s201.q4cdn.com/141608511/files/doc_presentations/2020/12/GTC-China_2020_FINAL-\(with-FLS\).pdf](https://s201.q4cdn.com/141608511/files/doc_presentations/2020/12/GTC-China_2020_FINAL-(with-FLS).pdf)
- [38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 19th Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, 2019.
- [39] Z. Duan, J. Yao, H. Liu, X. Liao, H. Jin, and Y. Zhang, "Revisiting log-structured merging for kv stores in hybrid memory systems," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.
- [40] A. Eisenman, D. Gardner, I. AbdelRahman, J. Axboe, S. Dong, K. Hazelwood, C. Petersen, A. Cidon, and S. Katti, "Reducing dram footprint with nvm in facebook," in *Proceedings of the Thirteenth EuroSys Conference (EuroSys)*, 2018.
- [41] A. Fazio, "Advanced technology and systems of cross point memory," in *Proceedings of the 65th International Electron Devices Meeting (IEDM)*, 2020.
- [42] D. Foley and J. Danskin, "Ultra-performance pascal gpu and nvlink interconnect," *IEEE Micro*, vol. 37, no. 2, pp. 7–17, 2017.
- [43] S. W. Fong, C. M. Neumann, and H.-S. P. Wong, "Phase-change memory—towards a storage-class memory," *IEEE Transactions on Electron Devices*, vol. 64, no. 11, pp. 4374–4385, 2017.
- [44] S. Franey and M. Lipasti, "Tag tables," in *Proceedings of the 21st International Symposium on High Performance Computer Architecture (HPCA)*, 2015.
- [45] D. Ganguly, "Uvm smart," 2019. [Online]. Available: https://github.com/DebashisGanguly/gpgpu-sim_UVMSmart
- [46] D. Ganguly, Z. Zhang, J. Yang, and R. Melhem, "Interplay between hardware prefetcher and page eviction policy in cpu-gpu unified virtual memory," in *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*, 2019.

- [47] D. Ganguly, Z. Zhang, J. Yang, and R. Melhem, "Adaptive page migration for irregular data-intensive applications under gpu memory oversubscription," in *Proceedings of the 32nd International Parallel and Distributed Processing Symposium (IPDPS)*, 2020.
- [48] P. Gera, H. Kim, P. Sao, H. Kim, and D. Bader, "Traversing large graphs on gpus with unified memory," *Proc. VLDB Endow.*, vol. 13, no. 7, p. 1119–1133, mar 2020.
- [49] A. Gholami, Z. Yao, S. Kim, M. W. Mahoney, and K. Keutzer, "Ai and memory wall," 2021. [Online]. Available: <https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8>
- [50] B. Gopireddy and J. Torrellas, "Designing vertical processors in monolithic 3d," in *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*, 2019.
- [51] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to gpu codes," in *Proceedings of the 1st Innovative Parallel Computing (InPar)*, 2012.
- [52] Y. Gu, W. Wu, Y. Li, and L. Chen, "Uvmbench: A comprehensive benchmark suite for researching unified virtual memory in gpus," in *International Conference on Scientific Computing*, 2021.
- [53] T. Haruta, T. Nakajima, J. Hashizume, T. Umebayashi, H. Takahashi, K. Taniguchi, M. Kuroda, H. Sumihiro, K. Enoki, T. Yamasaki, K. Ikezawa, A. Kitahara, M. Zen, M. Oyama, H. Koga, H. Tsugawa, T. Ogita, T. Nagano, S. Takano, and T. Nomoto, "4.6 a 1/2.3inch 20mpixel 3-layer stacked cmos image sensor with dram," in *Proceedings of the 62nd International Solid-State Circuits Conference (ISSCC)*, 2017.
- [54] A. Hay, K. Strauss, T. Sherwood, G. H. Loh, and D. Burger, "Preventing pcm banks from seizing too much power," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-44. Association for Computing Machinery, 2011, p. 186–195.
- [55] M. Hildebrand, J. T. Angeles, J. Lowe-Power, and V. Akella, "A case against hardware managed dram caches for nvram based systems," in *Proceedings of the 14th International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2021.
- [56] S. Hong, H. Choi, J. Park, Y. Bae, K. Kim, W. Lee, S. Lee, H. Lee, S. Cho, J. Ahn, S. Kim, T. Kim, M. Na, and S. Cha, "Extremely high performance, high density 20nm self-selecting cross-point memory for compute express link," in *2022 International Electron Devices Meeting (IEDM)*, 2022.
- [57] C.-C. Huang, R. Kumar, M. Elver, B. Grot, and V. Nagarajan, "C3d: Mitigating the numa bottleneck via coherent dram caches," in *Proceedings of the 49th International Symposium on Microarchitecture (MICRO)*, 2016.
- [58] C.-C. Huang and V. Nagarajan, "Atcache: Reducing dram cache latency via a small sram tag cache," in *Proceedings of the 23rd International Conference on Parallel Architecture and Compilation Techniques (PACT)*, 2014.
- [59] T.-H. Hung, Y.-M. Pan, and K.-N. Chen, "Stress issue of vertical connections in 3d integration for high-bandwidth memory applications," *Memories - Materials, Devices, Circuits and Systems*, vol. 4, p. 100024, 2023.
- [60] D. Ielmini and S. Ambrogio, "Emerging neuromorphic devices," *Nanotechnology*, vol. 31, no. 9, p. 092001, dec 2019.
- [61] J. Izraelevitz, J. Yang, L. Zhang, J. Kim, X. Liu, A. Memaripour, Y. J. Soh, Z. Wang, Y. Xu, S. R. Dulloor *et al.*, "Basic performance measurements of the intel optane dc persistent memory module," *arXiv preprint arXiv:1903.05714*, 2019.
- [62] H. Jang, Y. Lee, J. Kim, Y. Kim, J. Kim, J. Jeong, and J. W. Lee, "Efficient footprint caching for tagless dram caches," in *Proceedings of the 22nd International Symposium on High Performance Computer Architecture (HPCA)*, 2016.
- [63] V. Jatala, R. Dathathri, G. Gill, L. Hoang, V. K. Nandivada, and K. Pingali, "A study of graph analytics for massive datasets on distributed multi-gpus," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2020, pp. 84–94.
- [64] D. Jevdjic, G. H. Loh, C. Kaynak, and B. Falsafi, "Unison cache: A scalable and effective die-stacked dram cache," in *Proceedings of the 47th International Symposium on Microarchitecture (MICRO)*, 2014.
- [65] D. Jevdjic, S. Volos, and B. Falsafi, "Die-stacked dram caches for servers: Hit ratio, latency, or bandwidth? have it all with footprint cache," in *Proceedings of the 40th International Symposium on Computer Architecture (ISCA)*, 2013.
- [66] X. Jiang, N. Madan, L. Zhao, M. Upton, R. Iyer, S. Makineni, D. Newell, Y. Solihin, and R. Balasubramanian, "Chop: Adaptive filter-based dram caching for cmp server platforms," in *Proceedings of the 16th International Symposium on High-Performance Computer Architecture (HPCA)*, 2010.
- [67] H. Jun, J. Cho, K. Lee, H.-Y. Son, K. Kim, H. Jin, and K. Kim, "Hbm (high bandwidth memory) dram technology and architecture," in *2017 IEEE International Memory Workshop (IMW)*, 2017.
- [68] H. Jun, J. Cho, K. Lee, H.-Y. Son, K. Kim, H. Jin, and K. Kim, "Hbm (high bandwidth memory) dram technology and architecture," in *Proceedings of the 9th International Memory Workshop (IMW)*, 2017.
- [69] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "Accelwattch: A power modeling framework for modern gpus," in *Proceedings of the 54th International Symposium on Microarchitecture (MICRO)*, 2021.
- [70] S. Kannan, A. Gavrilovska, V. Gupta, and K. Schwan, "Heteroos: Os design for heterogeneous memory management in datacenter," in *Proceedings of the 44th International Symposium on Computer Architecture (ISCA)*, 2017.
- [71] F. Kaplan, C. De Vivero, S. Howes, M. Arora, H. Homayoun, W. Burleson, D. Tullsen, and A. K. Coskun, "Modeling and analysis of phase change materials for efficient thermal management," in *Proceedings of the 32nd International Conference on Computer Design (ICCD)*, 2014.
- [72] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *Proceedings of the 47th International Symposium on Computer Architecture (ISCA)*, 2020.
- [73] H. Kim, J. Sim, P. Gera, R. Hadidi, and H. Kim, "Batch-aware unified memory management in gpus for irregular workloads," in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [74] T. Kim, H. Choi, M. Kim, J. Yi, D. Kim, S. Cho, H. Lee, C. Hwang, E.-R. Hwang, J. Song, S. Chae, Y. Chun, and J.-K. Kim, "High-performance, cost-effective 2z nm two-deck cross-point memory integrated by self-align scheme for 128 gb scm," in *Proceedings of the 63rd International Electron Devices Meeting (IEDM)*, 2018.
- [75] W. Kim, M. BrightSky, T. Masuda, N. Sosa, S. Kim, R. Bruce, F. Carta, G. Fraczak, H. Y. Cheng, A. Ray, Y. Zhu, H. L. Lung, K. Suu, and C. Lam, "Ald-based confined pcm with a metallic liner toward unlimited endurance," in *2016 IEEE International Electron Devices Meeting (IEDM)*, 2016.
- [76] W. Kim, R. Bruce, T. Masuda, G. Fraczak, N. Gong, P. Adusumilli, S. Ambrogio, H. Tsai, J. Bruley, J.-P. Han, M. Longstreet, F. Carta, K. Suu, and M. BrightSky, "Confined pcm-based analog synaptic devices offering low resistance-drift and 1000 programmable states for deep learning," in *2019 Symposium on VLSI Technology*, 2019.
- [77] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [78] Y. Kim, H. Kim, and W. J. Song, "Nomad: Enabling non-blocking os-managed dram cache via tag-data decoupling," in *Proceedings of the 29th International Symposium on High Performance Computer Architecture (HPCA)*, 2023.
- [79] Y. Kim, J. Lee, J.-E. Jo, and J. Kim, "Gpudmm: A high-performance and memory-oblivious gpu architecture using dynamic memory management," in *Proceedings of the 20th International Symposium on High Performance Computer Architecture (HPCA)*, 2014.
- [80] Y. Ko, H. Kim, and H. Han, "Escalating memory accesses to shared memory by profiling reuse," in *Proceedings of the 10th International Conference on Ubiquitous Information Management and Communication (IMCOM)*, 2016.
- [81] D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G.-M. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, V. Kornijuk, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Lee, D. Ko, Y. Jun, I. Kim, C. Song, I. Kim, C. Park, S. Kim, C. Jeong, E. Lim, D. Kim, J. Jang, I. Park, J. Chun, and J. Cho, "A 1ynm 1.25v 8gb 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep learning application," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 291–302, 2023.
- [82] D. Kwon, H. S. Jeong, J. Choi, W. Kim, J. W. Kim, J. Yoon, J. Choi, S. Lee, H. N. Rie, J.-i. Lee, J. Lee, T. Jang, J. Kim, S. Kang, J. Shin, Y. Loh, C. Y. Lee, J. Woo, H. Yu, C. Bae, R. Oh, Y.-s. Sohn, C. Yoo, and J. Lee, "28.7 a 1.1v 6.4gb/s/pin 24-gb ddr5 sdram with a highly-accurate duty corrector and nbt-tolerant dll," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023.

- [83] B. C. Lee, E. Ipek, O. Mutlu, and D. Burger, "Architecting phase change memory as a scalable dram alternative," in *Proceedings of the 36th international symposium on Computer architecture (ISCA)*, 2009.
- [84] H. Lee, H. Kim, S. Shim, S. Lee, D. Hong, H.-J. Lee, and H. Kim, "Pmcsm: An accurate phase-change memory controller simulator and its performance analysis," in *Proceedings of the 15th International Symposium on Performance Analysis of Systems and Software (ISPASS)*.
- [85] N. Lee, "Expanding the boundaries of ai revolution: An in-depth study of hbm (presented by sk hynix)," NVIDIA GPU Technology Conference, 2018. [Online]. Available: <https://www.nvidia.com/en-us/on-demand/session/gtc-siliconvalley2018-s8949/>
- [86] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for pim based on commercial dram technology : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
- [87] S. Lee, K. Lee, M. Sung, M. Alian, C. Kim, W. Cho, R. Oh, S. O, J. H. Ahn, and N. S. Kim, "3d-xpath: High-density managed dram architecture with cost-effective alternative paths for memory transactions," in *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2018.
- [88] Y. Lee, J. Kim, H. Jang, H. Yang, J. Kim, J. Jeong, and J. W. Lee, "A fully associative, tagless dram cache," in *Proceedings of the 42nd International Symposium on Computer Architecture (ISCA)*, 2015.
- [89] Y. S. Lee, K. M. Kim, J. H. Lee, J. H. Choi, and S. W. Chung, "A high-performance processing-in-memory accelerator for inline data deduplication," in *Proceedings of the 37th International Conference on Computer Design (ICCD)*, 2019.
- [90] C. Li, R. Ausavarungnirun, C. J. Rossbach, Y. Zhang, O. Mutlu, Y. Guo, and J. Yang, "A framework for memory oversubscription management in graphics processing units," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [91] Y. Li and M. Gao, "Baryon: Efficient hybrid memory management with compression and sub-blocking," in *Proceedings of the 29th International Symposium on High Performance Computer Architecture (HPCA)*, 2023.
- [92] Y. Li, A. Phanishayee, D. Murray, J. Tarnawski, and N. S. Kim, "Harmony: Overcoming the hurdles of gpu memory capacity to train massive dnn models on commodity servers," *Proc. VLDB Endow.*, vol. 15, no. 11, p. 2747–2760, jul 2022.
- [93] J. Liu, B. Jaiyen, Y. Kim, C. Wilkerson, and O. Mutlu, "An experimental study of data retention behavior in modern dram devices: Implications for retention time profiling mechanisms," in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*, 2013.
- [94] L. Liu, S. Yang, L. Peng, and X. Li, "Hierarchical hybrid memory management in os for tiered memory systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 10, pp. 2223–2236, 2019.
- [95] G. Loh and M. D. Hill, "Supporting very large dram caches with compound-access scheduling and missmap," *IEEE Micro*, vol. 32, no. 3, pp. 70–78, 2012.
- [96] G. H. Loh, N. E. Jerger, A. Kannan, and Y. Eckert, "Interconnect-memory challenges for multi-chip, silicon interposer systems," in *Proceedings of the 1st International Symposium on Memory Systems (MEMSYS)*, 2015.
- [97] T. Lu, C. Serafy, Z. Yang, S. K. Samal, S. K. Lim, and A. Srivastava, "Tsv-based 3-d ics: Design methods and tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 36, no. 10, pp. 1593–1619, 2017.
- [98] S. Mach, F. Schuiki, F. Zaruba, and L. Benini, "Fpnew: An open-source multiformat floating-point unit architecture for energy-proportional transprecision computing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 4, pp. 774–787, 2020.
- [99] J. Macri, "Amd's next generation gpu and high bandwidth memory architecture: Fury," in *2015 IEEE Hot Chips 27 Symposium (HCS)*, 2015.
- [100] J. Meng, K. Kawakami, and A. K. Coskun, "Optimizing energy efficiency of 3-d multicore systems with stacked dram under power and thermal constraints," in *Proceedings of the 49th Design Automation Conference (DAC)*, 2012.
- [101] M. R. Meswani, S. Blagodurov, D. Roberts, J. Slice, M. Ignatowski, and G. H. Loh, "Heterogeneous memory architectures: A hw/sw approach for mixing die-stacked and off-package memories," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, 2015.
- [102] J. Meza, J. Chang, H. Yoon, O. Mutlu, and P. Ranganathan, "Enabling efficient and scalable hybrid memories using fine-granularity dram cache management," *IEEE Computer Architecture Letters*, vol. 11, no. 2, pp. 61–64, 2012.
- [103] P. Micikevicius, "Multi-gpu programming," NVIDIA GPU Technology Conference, 2012.
- [104] T. P. Morgan, "The era of big memory is upon us," September 2020. [Online]. Available: <https://www.nextplatform.com/2020/09/23/the-era-of-big-memory-is-upon-us/>
- [105] T. P. Morgan, "THE THIRD TIME CHARM OF AMD'S INSTINCT GPU," June 2023. [Online]. Available: <https://www.nextplatform.com/2023/06/14/the-third-time-charm-of-amds-instinct-gpu/>
- [106] L. Nai, Y. Xia, I. G. Tanase, H. Kim, and C.-Y. Lin, "Graphbig: Understanding graph computing in the context of industrial solutions," in *Proceedings of the 28th International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [107] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.
- [108] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained dram: Energy-efficient dram for extreme bandwidth systems," in *the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017.
- [109] Y. Pan, Y. Wang, Y. Wu, C. Yang, and J. D. Owens, "Multi-gpu graph analytics," in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2017.
- [110] S. Pandey, A. K. Kamath, and A. Basu, "Gpm: Leveraging persistent memory from a gpu," in *Proceedings of the 27th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022.
- [111] M. K. Qureshi, M. M. Franceschini, L. A. Lastras-Montano, and J. P. Karidis, "Morphable memory system: A robust architecture for exploiting multi-level phase change memories," in *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*, 2010.
- [112] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Set-dueling-controlled adaptive insertion for high-performance caching," *IEEE Micro*, vol. 28, no. 1, pp. 91–98, 2008.
- [113] M. K. Qureshi and G. H. Loh, "Fundamental latency trade-off in architecting dram caches: Outperforming impractical sram-tags with a simple and practical design," in *Proceedings of the 45th International Symposium on Microarchitecture (MICRO)*, 2012.
- [114] M. K. Qureshi, V. Srinivasan, and J. A. Rivers, "Scalable high performance main memory system using phase-change memory technology," in *Proceedings of the 36th International Symposium on Computer Architecture (ISCA)*, 2009.
- [115] Z. Qureshi, V. S. Mailthody, S. W. Min, I.-H. Chung, J. Xiong, and W. mei Hwu, "Tearing down the memory wall," Semiconductor Corporation TechCon, 2020.
- [116] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of the 16th Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016.
- [117] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, "Vdnn: Virtualized deep neural networks for scalable, memory-efficient neural network design," in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.
- [118] M. Rhu, M. Sullivan, J. Leng, and M. Erez, "A locality-aware memory hierarchy for energy-efficient gpu architectures," in *Proceedings of the 46th International Symposium on Microarchitecture (MICRO)*, 2013.
- [119] J. Roach, "To cool datacenter servers, microsoft turns to boiling liquid," April 2021. [Online]. Available: <https://news.microsoft.com/source/features/innovation/datacenter-liquid-cooling/>
- [120] N. Sakharnykh, "Everything you need to know about unified memory," NVIDIA GPU Technology Conference, 2018.
- [121] C. Shao, J. Guo, P. Wang, J. Wang, C. Li, and M. Guo, "Oversubscribing gpu unified virtual memory: Implications and suggestions," in *Proceedings of the 12nd International Conference on Performance Engineering (ICPE)*, 2022.

- [122] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
- [123] J. Sim, A. R. Alameldeen, Z. Chishti, C. Wilkerson, and H. Kim, "Transparent hardware management of stacked dram as part of memory," in *Proceedings of the 47th International Symposium on Microarchitecture (MICRO)*, 2014.
- [124] J. Sim, G. H. Loh, H. Kim, M. OConnor, and M. Thottethodi, "A mostly-clean dram cache for effective hit speculation and self-balancing dispatch," in *Proceedings of the 45th International Symposium on Microarchitecture (MICRO)*, 2012.
- [125] A. Sodani, R. Gramunt, J. Corbal, H.-S. Kim, K. Vinod, S. Chinthamani, S. Hutsell, R. Agarwal, and Y.-C. Liu, "Knights landing: Second-generation intel xeon phi product," *Ieee micro*, vol. 36, no. 2, pp. 34–46, 2016.
- [126] S. Somogyi, T. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos, "Spatial memory streaming," in *Proceedings of the 33rd International Symposium on Computer Architecture (ISCA)*, 2006.
- [127] Y. Song, W.-H. Kim, S. K. Monga, C. Min, and Y. I. Eom, "Prism: Optimizing key-value store for modern heterogeneous storage devices," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.
- [128] K. Stern, N. Wainstein, Y. Keller, C. M. Neumann, E. Pop, S. Kvatinisky, and E. Yalon, "Uncovering phase change memory energy limits by sub-nanosecond probing of power dissipation dynamics," *Advanced Electronic Materials*, vol. 7, no. 8, p. 2100217, 2021.
- [129] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," *Center for Reliable and High-Performance Computing*, vol. 127, p. 27, 2012.
- [130] G. Thomas-Collignon and V. Mehta, "Optimizing cuda applications for nvidia a100 gpu," NVIDIA GPU Technology Conference, 2020.
- [131] D. Ustiugov, A. Daglis, J. Picorel, M. Sutherland, E. Bugnion, B. Falsafi, and D. Pnevmatikatos, "Design guidelines for high-performance scm hierarchies," in *Proceedings of the 4th International Symposium on Memory Systems (MEMSYS)*, 2018.
- [132] Z. Wang, "Microsystems using three-dimensional integration and tsv technologies: Fundamentals and applications," *Microelectronic Engineering*, vol. 210, pp. 35–64, 2019.
- [133] Z. Wang, X. Liu, J. Yang, T. Michailidis, S. Swanson, and J. Zhao, "Characterizing and modeling non-volatile memory systems," in *Proceedings of the 53rd International Symposium on Microarchitecture (MICRO)*, 2020.
- [134] M. Webb, "Annual update on emerging memories 2020," Flash Memory Summit, 2020.
- [135] J. Wu, Y. Chen, W. S. Khwa, S. M. Yu, T. Y. Wang, J. Tseng, Y. Chih, and C. H. Diaz, "A 40nm low-power logic compatible phase change memory technology," in *Proceedings of the 63rd International Electron Devices Meeting (IEDM)*, 2018.
- [136] L. Xiang, X. Zhao, J. Rao, S. Jiang, and H. Jiang, "Characterizing the performance of intel optane persistent memory: A close look at its on-dimm buffering," in *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*, 2022.
- [137] F. Xiong, E. Yalon, A. Behnam, C. Neumann, K. Grosse, S. Deshmukh, and E. Pop, "Towards ultimate scaling limits of phase-change memory," in *2016 IEEE International Electron Devices Meeting (IEDM)*, 2016.
- [138] Z. Yan, D. Lustig, D. Nellans, and A. Bhattacharjee, "Nimble page management for tiered memory systems," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [139] D. Yang, J. Liu, J. Qi, and J. Lai, "Wholegraph: A fast graph neural network training framework with multi-gpu distributed shared memory architecture," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2022.
- [140] L. Yavits, L. Orosa, S. Mahar, J. D. Ferreira, M. Erez, R. Ginosar, and O. Mutlu, "Wolfram: Enhancing wear-leveling and fault tolerance in resistive memories using programmable address decoders," in *Proceedings of the 38th International Conference on Computer Design (ICCD)*, 2020.
- [141] J. Yi, M. Kim, J. Seo, N. Park, S. Lee, J. Kim, G. Do, H. Jang, H. Koo, S. Cho, S. Chae, T. Kim, M.-H. Na, and S. Cha, "The chalcogenide-based memory technology continues: beyond 20nm 4-deck 256gb cross-point memory," in *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, 2023.
- [142] H. Yoon, J. Meza, R. Ausavarungrun, R. A. Harding, and O. Mutlu, "Row buffer locality aware caching policies for hybrid memories," in *Proceedings of the 30th International Conference on Computer Design (ICCD)*, 2012.
- [143] V. Young, Z. A. Chishti, and M. K. Qureshi, "Tictoc: Enabling bandwidth-efficient dram caching for both hits and misses in hybrid memory systems," in *Proceedings of the 37th International Conference on Computer Design (ICCD)*, 2019.
- [144] V. Young, C. Chou, A. Jaleel, and M. Qureshi, "Accord: Enabling associativity for gigascale dram caches by coordinating way-install and way-prediction," in *Proceedings of the 45th International Symposium on Computer Architecture (ISCA)*, 2018.
- [145] V. Young, A. Jaleel, E. Bolotin, E. Ebrahimi, D. Nellans, and O. Villa, "Combining hw/sw mechanisms to improve numa performance of multi-gpu systems," in *Proceedings of the 51st International Symposium on Microarchitecture (MICRO)*, 2018.
- [146] V. Young, P. J. Nair, and M. K. Qureshi, "Dice: Compressing dram caches for bandwidth and capacity," in *Proceedings of the 44th International Symposium on Computer Architecture (ISCA)*, 2017.
- [147] V. Young and M. K. Qureshi, "To update or not to update?: Bandwidth-efficient intelligent replacement policies for DRAM caches," in *Proceedings of the 37th International Conference on Computer Design (ICCD)*, 2019.
- [148] X. Yu, C. J. Hughes, N. Satish, O. Mutlu, and S. Devadas, "Banshee: Bandwidth-efficient dram caching via software/hardware cooperation," in *Proceedings of the 50th International Symposium on Microarchitecture (MICRO)*, 2017.
- [149] D. Zhang, N. Jayasena, A. Lyashevsky, J. L. Greathouse, L. Xu, and M. Ignatowski, "Top-pim: Throughput-oriented programmable processing in memory," in *Proceedings of the 23rd international symposium on High-performance parallel and distributed computing (HPDC)*, 2014.
- [150] J. Zhang and M. Jung, "Zng: Architecting gpu multi-processors with new flash for scalable data analysis," in *Proceedings of the 47th International Symposium on Computer Architecture (ISCA)*, 2020.
- [151] J. Zhang and M. Jung, "Ohm-gpu: Integrating new optical network and heterogeneous memory into gpu multi-processors," in *Proceedings of the 54th International Symposium on Microarchitecture (MICRO)*, 2021.
- [152] J. Zhang, M. Kwon, H. Kim, H. Kim, and M. Jung, "Flashgpu: Placing new flash next to gpu cores," in *Proceedings of the 56th Design Automation Conference (DAC)*, 2019.
- [153] R. Zhang, M. R. Stan, and K. Skadron, "Hotspot 6.0: Validation, acceleration and extension," *University of Virginia, Tech. Rep.*, 2015.
- [154] W. Zhang and T. Li, "Exploring phase change memory and 3d die-stacking for power/thermal friendly, fast and durable memory architectures," in *Proceedings of the 18th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2009.
- [155] J. Zhao and Y. Xie, "Optimizing bandwidth and power of graphics memory with hybrid memory technologies and adaptive data migration," in *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, 2012.
- [156] T. Zheng, D. Nellans, A. Zulfiqar, M. Stephenson, and S. W. Keckler, "Towards high performance paged memory for gpus," in *Proceedings of the 22nd International Symposium on High Performance Computer Architecture (HPCA)*, 2016.